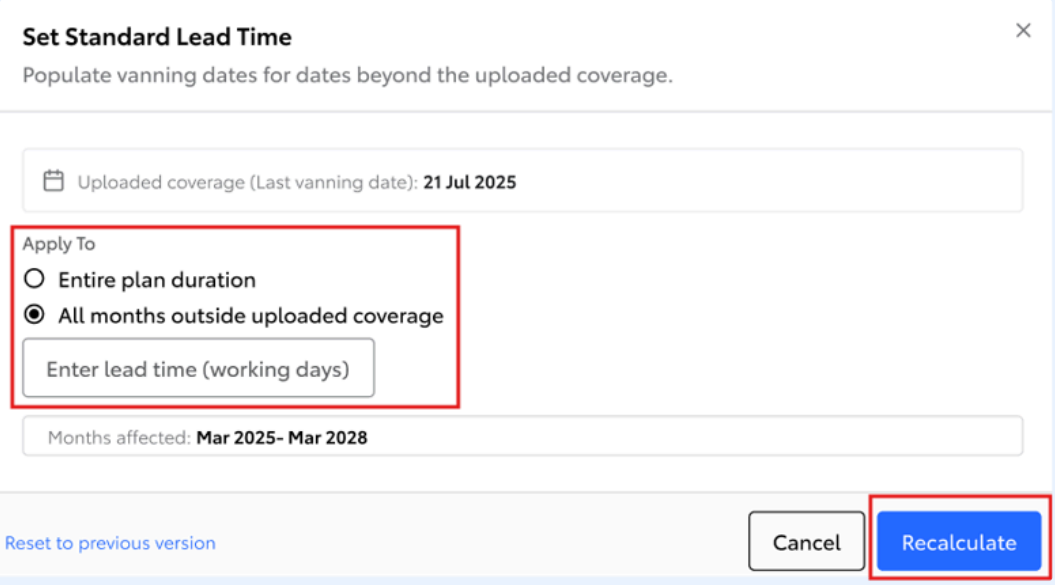
SPPL-196: Apply Standard Lead Time Post API technical workflow

Jira Ticket: SPPL-196: BE | Create technical workflow for Vanning Lead Time Get & Post APIs on parameter inputs page including DB details ACCEPTED

This page provides comprehensive information about the POST API for Apply Standard Vanning Lead Time on the Input params page, to generate & apply order dates based on user provided lead time.



Supply Planning POST Standard Lead Time API Details:

1. **Hosted URL**: https://api.spln.cube<env>.toyota.com/<env>
2. **API Name:** <HOSTED_URL>/v/1/1/sp/apply-standard-lead-time
3. **Path Param**: N/A
4. **Query Param**: N/A
5. **Method:** POST
6. **Authorization**: Bearer token (**required**)
7. **Lambda Name**: spln-<env>-apply-standard-lead-time-v1-1
8. **API Request Body**:

```
{
  "scenarioId": "uniqueScenarioId",
  "entirePlanDuration": true,
  "userEmail": "user@toyota.com",
  "vanningCenter": "RAV4",
  "leadTime": 14
}
```

9. **API Response:**

- **Success response with status 200:**

```
{
  "message": "Successfully updated data."
}
```

- **Error Response:**
  - **any internal server error occurred with status 500:**

```
{
  "errorMessage": "Internal Server Error"
}
```

  - **any unauthorized error occurred with status 401 (Managed by the AWS API Gateway integrated Custom Authorizer):**

```
{
  "message": "Unauthorized"
}
```

  - **If request is in-valid with status 400:**

```
{
  "errorMessage": [<"ValidationError: validation error message”>]
}
```

  - **List of possible validation error message:**
    1. "ValidationError: Request body cannot be empty.”
    2. "ValidationError: scenarioId is required and must be a uuid."
    3. "ValidationError: entirePlanDuration must be either true or false."
    4. "ValidationError: userEmail is required and must be a string."
    5. “ValidationError: vanningCenter is required and must be a string."
    6. "ValidationError: Scenario doesn't exist."
    7. "ValidationError: Invalid userEmail."
    8. "ValidationError: leadTime is required and must be a non-negative number not more than 99."
    9. “ValidationError: No TMC working day calendar data found for scenarioId <scenarioId> and vanningCenter <vanningCenter>”
    10. “ValidationError: Vanning lead time data already exists for the entire scenario duration. No update was performed.“

10. **Table:**
    a. **scenarios**
    b. **tmc_working_day_calendar**
    c. **standard_lead_time**

d. **vanning_lead_time**

e. **scenario_step_status**

SP Apply Standard Vanning Lead Time Post API Workflow:

- **Technical workflow:**
  - Description:
    i. The Supply Planning user logs in to the application.
    ii. After the user provides creates a scenario, opens it and applies Standard Vanning Lead Time, a request is sent to the SP Post Standard Vanning Lead Time endpoint: ***sp/apply-standard-lead-time*** with request body:

```
{
  "scenarioId": "uniqueScenarioId",
  "entirePlanDuration": true,
  "userEmail": "user@toyota.com",
  "vanningCenter": "RAV4",
  "leadTime": 14
}
```

    iii. After successful authentication, the **spln-*apply-standard-lead-time*-v1-1** lambda is triggered by the SP scenario apply generic Vanning Lead Time endpoint: ***sp/apply-standard-lead-time***. If authentication fails, an error with status 401 (*refer: error response-**any unauthorized error occurred with status 401***) will be returned and sent to client.
    iv. After user authentication has been confirmed, the **spln-*apply-standard-lead-time*-v1-1** lambda will perform validation on the request payload. If payload is valid will proceed to next step else throw validation error (*refer: error response**: Section if request is in-valid with status 400 and List of possible validation error message***). Below queries are used to check if provided scenario is valid:

```
select * from supply_planning.scenarios where scenario_id = :scenarioId and
is_active = true;
```

    v. After the request payload is successfully validated, the **sp-*apply-standard-lead-time*-v1-1** lambda execute logic to generate vanning lead time data based on the inputs provided for either entire timeframe or partial timeframe (remaining timeframe for which data doesn't exist in file).
    vi. Fetch TMC Working Day Calendar data for given scenarioId and vanningCenter in order to identify working & non-working days using below query:

```
select tmc_date         as "prodDate",
day_of_week       as "dayOfWeek",
is_working_day    as "isWorkingDay",
work_day_percentage as "workDayPercentage",
```

```
week_number              as "week"
from supply_planning.tmc_working_day_calendar
where scenario_id = ${body.scenarioId}::uuid
and vanning_center = ${body.vanningCenter}
order by tmc_date;
```

vii. In case of no TMC Working Day Calendar data for given scenarioId and vanningCenter, throw validation error (9) as atleast prefilled data must exist in the ***tmc_working_day_calendar*** table.

viii. If we have partial timeframe request and the complete timeframe vanning lead time data already exists in the ***vanning_lead_time*** table, throw validation error (10).

ix. Below query to post scenario Standard Vanning Lead Time data into the ***standard_lead_time*** & **vanning_lead_time** table.

```
-- if user selects partial timeframe, fetch existing data from vanning_lead_time table
-- to identify remaining timeframe else use scenario timeframe from query response (iv)
SELECT * from supply_planning.vanning_lead_time where scenario_id = :scenarioId and is_active = true;

-- Upsert standard_lead_time table data
await tx.$executeRaw(
  Prisma.sql`
    INSERT INTO supply_planning.standard_lead_time (
      scenario_id,
      vanning_center,
      lead_time,
      apply_to_all,
      created_by,
      created_date_timestamp
    )
    VALUES (
      ${scenarioId}::uuid,
      ${vanningCenter}::text,
      ${leadTime}::integer,
      ${entirePlanDuration}::boolean,
      ${userEmail}::text,
      NOW()
    )
    ON CONFLICT (scenario_id, vanning_center)
    DO UPDATE SET
      lead_time = EXCLUDED.lead_time,
      apply_to_all = EXCLUDED.apply_to_all,
      updated_by = ${userEmail}::text,
      last_updated_timestamp = NOW()`
);
-- vanning_lead_time
await prisma.$executeRaw`
INSERT INTO supply_planning.vanning_lead_time (
  scenario_id,
  namc,
  vanning_center,
  vanning_date,
  vanning_day,
  tmc_working,
  order_date,
  order_day,
  lead_time,
  created_by
)
```

```
SELECT
  ${scenarioId}::uuid AS scenario_id,
  v.namc,
  v.vanning_center,
  v.vanning_date,
  v.vanning_day,
  v.tmc_working,
  v.order_date,
  v.order_day,
  v.lead_time,
  ${userEmail}::text AS created_by
FROM (
  VALUES
    ${Prisma.join(
      input.map(item => Prisma.sql`(
        ${item.namc}::text,
        ${item.vanningCenter}::text,
        ${item.vanningDate}::date,
        ${item.vanningDay}::text,
        ${item.isWorkingDay}::boolean,
        ${item.orderDate}::date,
        ${item.orderDay}::text,
        ${item.leadTime}::integer
      )`)
    )}
) AS v(
  namc,
  vanning_center,
  vanning_date,
  vanning_day,
  tmc_working,
  order_date,
  order_day,
  lead_time
)
ON CONFLICT (scenario_id, vanning_center, vanning_date)
DO UPDATE SET
  tmc_working = EXCLUDED.tmc_working,
  order_date  = EXCLUDED.order_date,
  order_day = EXCLUDED.order_day,
  lead_time = EXCLUDED.lead_time,
  updated_by = ${userEmail}::text,
  last_updated_timestamp = NOW()
`;
```

x. Update the step & scenario status to “In Progress”. Only update scenario status to “In Progress” if its not already using scenarioData from query (iv) response.

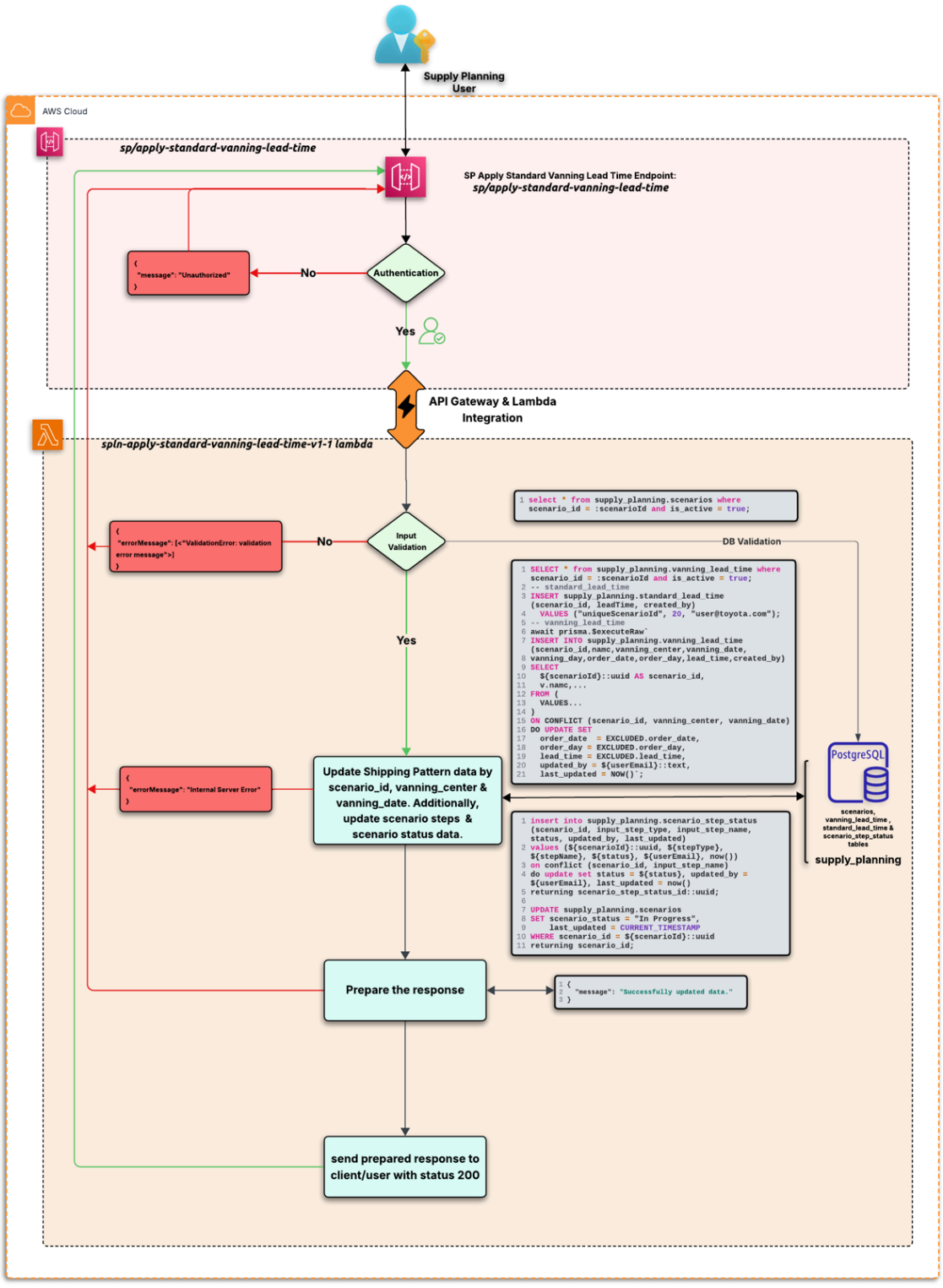
```
insert into supply_planning.scenario_step_status (scenario_id,
input_step_type, input_step_name, status, created_by, created_date_timestamp)
values (${scenarioId}::uuid, ${stepType}, ${stepName}, ${status},
${userEmail}, now())
on conflict (scenario_id, input_step_name)
do update set status = ${status}, updated_by = ${userEmail},
last_updated_timestamp = now()
returning scenario_step_status_id::uuid;

UPDATE supply_planning.scenarios
SET scenario_status = ${scenarioStatus},
    updated_by = ${userEmail},
    last_updated_timestamp = CURRENT_TIMESTAMP
WHERE scenario_id = ${scenarioId}::uuid
returning scenario_id;
```

xi. After updating the Vanning Lead Time data, return the below response to the client with a status code of 200:

```
{
  "message": "Successfully updated data."
}
```

xii. If there is an error while updating the data, an “Internal Server Error” with a status code of 500(*refer: error response-**any internal server error occurred with status 500***) will be returned. Otherwise, a successful response with a status code of 200 will be sent.



SP Apply Standard Lead Time Post API Technical workflow

Note:

We will be storing both working & non-working days to vanning_lead_time table.

vanning_date - leadTime (workingDays) = order_date

Queries: